Assignment Instructions

1. This is a **group assignment for CSC 665** (optional CSC 865). You are responsible for forming your own group of exactly two students.

   - Please **form your pair on Canvas by Thursday, February 12** by joining a group under *People* → *assignment1-groups*.
     Individual submissions for CSC 665 will not be accepted.

   Here are the general rules for teams:

   (a) You should work on the whole assignment together. If you decide to work separately, review and discuss your solutions afterward and consolidate them into one joint submission.

   (b) You should **not** just partition the work between you.

   (c) You should upload only one submission to Canvas.

2. Written Parts: [Problem I: Part 2, Part 3 Qs 3-4 and Problem II Part 2]

   - Please type your answers and hand them in as a **PDF** file through Canvas.
   - Your full name(s), as they appear on Canvas, should be included at the top of the PDF.

3. Coding Parts: [Problem I: Part 1, Part 3 Q 1 and Problem II]

   - All source files should be compressed into a *ZIP* archive.
   - Submit the *ZIP* file through Canvas.
   - Be sure your programs execute properly. Failure to do so will mean that your program is not graded.
     Note: Assignments are graded from the terminal.
   - File naming format: `firstLastname1_firstLastname2_asmt1.zip` for teams and `<firstLastName>_asmt1.zip` for individual submission (CSC 865 only).
   - You should use the code provided on Canvas in `assignment1_code.zip`

4. If you use **GenAI**, include a short paragraph stating:

   - which questions you used it for and
   - what changes you made to the output (minor or substantial).

5. Please refer to the university integrity policy on the syllabus.

Assignment Goals and Objectives

1. Implement an agent-based search problem
2. Compare search algorithms
3. Compute and analyze key complexity measures
4. Create and interpret runtime plots
5. Build an AI agent for a minimax game
6. Analyze the performance and scaling behavior of algorithms

Problem I

The Three-Jug Water Problem

We have discussed the two jugs of water problem in class. Now you will implement a slightly more complex and interesting version of the same problem. Instead of just two jugs, you are given three jugs with different maximum capacities, c_1, c_2 and c_3. Initially, all three jugs are empty. You can perform the following operations any number of times:

- Fill a jug completely
- Empty a jug
- Pour water from jug i into jug j until jug j is full or jug i is empty

Unlike with the two-jug problem, the goal here is to reach a specific target state where each jug contains a desired amount of water.

You may only pour from one jug into one other jug at a time.

There are four files in this project: `runner.py`, `the3jugs.py`, `solvers.py` and `test_cases.json`.

- `the3jugs.py` contains the logic for modeling the search problem.
- `solvers.py` contains implementations of the search algorithms used to solve the problem.
- `runner.py` reads test cases from a JSON file and runs all search algorithms. However, it will only be functional once you complete the implementation of the search algorithms in `solvers.py`.
- `test_cases.json` contains a list of test cases for evaluating your program and answering questions in Parts 2 and 3. The results shown in Part 2 are based on these cases.

Please do not modify the following functions in `runner.py`: `read_cases_from_json` and `main`

Part 1 – Implementation

1. In `the3jugs.py`, implement
   (a) `def succ(self, state, action)`
   (b) `def actions(self, state)`
2. In `solvers.py`, implement
   (a) `BFSSearch`
   (b) `DFSSearch`

   The algorithms should find a sequence of actions leading from the initial empty state to the target state. Both algorithms should **return a dictionary** similar to `BacktrackingSearch`

Part 2 – Complexity Analysis (GenAI not allowed for this part)

The following is a plot of the time it takes to run each of these algorithms for any given sum of capacities (i.e., $jug1_{capacity} + jug2_{capacity} + jug3_{capacity}$)

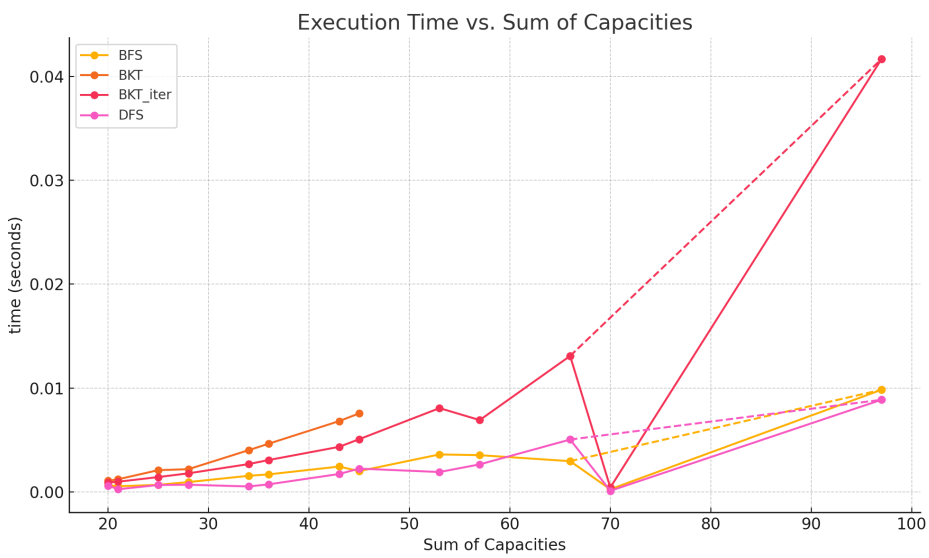

Figure 1: Comparison of BFS, DFS, BKT (i.e. *BacktrackingSearch*), and BKT_iter (i.e. *BacktrackingSearchIterative*) runtimes for the 3-jugs problem across different capacity sums. Solid lines connect actual measured runtimes. Dashed line segments highlight the runtime trend between sums 66 and 97, drawn as if the data point at $sum = 70$ did not exist, to illustrate continuity and emphasize performance progression in that range.

1. Why is there no data for the recursive `BacktrackingSearch` (BKT) beyond $sum = 45$?
2. What could explain the significant dip in execution time at $sum = 70$ across all algorithms?

Part 3 – Let's Dig Deeper (light help from GenAI only*)

** The goal is to build intuition for why things work the way they do – by modifying the code yourself (or with minimal help from GenAI), you will better understand how the algorithms work.*

In order to better understand the scaling behavior of these algorithms, we need more information. We will only look at DFS and BFS. If you are curious, feel free to include `BacktrackingSearch` and `BacktrackingSearchIterative`.

1. For each algorithm, modify your implementation to compute the average branching factor (b), the maximum depth of the search space (D), and the depth of the shallowest solution (d).
2. Run your program on the provided test file
3. Generate two plots, one for BFS and one for DFS. Use the Sum of Capacities on the x-axis. Plot the branching factor and depth (d for BFS and D for DFS) on the left y-axis, and add execution time on a secondary y-axis (right). Figure 2 provides a template for reference.
4. Do the plots confirm your answer to Part 2 question 2? If your answer was different, keep it — I'd love to see it. Use these plots to explain the dip in execution time at $sum = 70$.

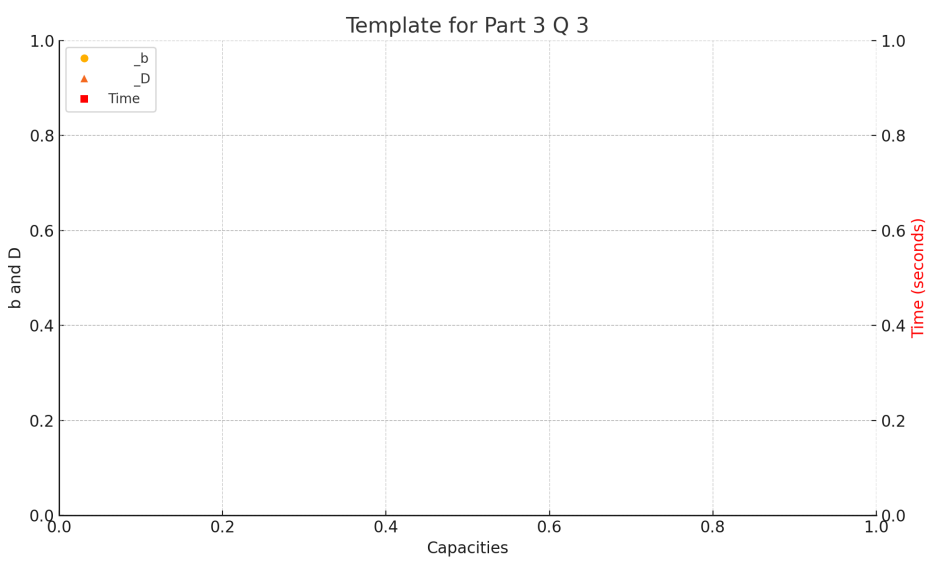

Figure 2: Template for Part 3 question 3.

Problem II

Coins in a line

This strategy game consists of a line of **n coins**, each with a positive integer value (e.g., [3, 9, 1, 2]). Two players take turns picking coins from either end of the line. On each turn, a player may **pick one or two consecutive coins from either the left or the right end** until there are no coins left.

The goal for each player is to **maximize the total value of the coins** they collect.

Because choices made early in the game affect the remaining options and potential outcomes, players must reason about future moves and anticipate their opponent's actions. We will use `minimax` to implement a program that plays the game optimally.

Part 1 – Implementation

There are two main files in this project: `runner.py` and `coinline.py`. `coinline.py` contains the logic for playing the game, and for making optimal moves. `runner.py` is fully implemented, and contains all of the code to run the graphical interface for the game. However, it will only be functional once you complete the implementation of all the functions in `coinline.py`. You may try your implementation by playing against the AI. **Please do not modify** `runner.py`. In `coinline.py`, implement the following:

1. player
2. actions
3. succ
4. winner
5. terminal
6. utility
7. minimax

Part 2 – Analysis

The number of coins in the line is set to 10 in the starter code. After you successfully run the program, increase the number of coins by 5 each time using the variable `NUM_COINS` in `runner.py`. The coins will not be clearly visible, but you should still be able to hit the buttons.

1. What happens as you increase the number of coins?
2. Why do you think that is?
3. Briefly describe how you would fix this problem. (No implementation is required for this question)